

1 Question 1

1. The output feature vector $z_i^{(t+1,k)}$ for a single attention head k at node i is computed as a weighted sum of the linearly transformed input features of its neighbors $j \in \mathcal{N}(i)$, followed by a non-linear activation function. The algebraic equation is given by:

$$z_i^{(t+1,k)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}^{(k)} z_j^{(t)} \right) \quad (1)$$

Where:

- $\mathcal{N}(i)$ represents the set of neighbors of node i (including node i).
- $\mathbf{W}^{(k)}$ is the learnable weight matrix associated with the k -th attention head.
- $\alpha_{ij}^{(k)}$ represents the normalized attention coefficients computed by the k -th head, indicating the importance of node j 's features to node i .
- $z_j^{(t)}$ is the input feature vector of neighbor j at layer t .
- σ is the activation function.

2. To obtain the final representation $z_i^{(t+1)}$, the output features from all K independent attention heads are concatenated. The equation is:

$$z_i^{(t+1)} = \left\|_{k=1}^K z_i^{(t+1,k)} \right\| = \left\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}^{(k)} z_j^{(t)} \right) \right\| \quad (2)$$

Dimensionality:

Since each individual head k produces an output vector $z_i^{(t+1,k)}$ of dimension F'_{out} , and there are K heads concatenated together, the total dimensionality of the final feature vector $z_i^{(t+1)}$ is:

$$D = K \times F'_{out}$$

3. To derive the total number of learnable parameters, we consider the parameters required for a single attention head k and then sum over all K heads. The learnable parameters consist of the weight matrix $\mathbf{W}^{(k)}$ and the attention vector $\mathbf{a}^{(k)}$.

- $\mathbf{W}^{(k)}$: This matrix performs a linear transformation from the input dimension F_{in} to the output dimension F'_{out} .

$$\dim(\mathbf{W}^{(k)}) = F'_{out} \times F_{in} \implies F_{in} \cdot F'_{out} \text{ parameters}$$
- $\mathbf{a}^{(k)}$: The attention mechanism operates on the concatenation of the transformed features of two nodes (source and target), i.e., a vector of dimension $F'_{out} + F'_{out} = 2F'_{out}$. The attention vector projects this concatenation to a scalar.

$$\dim(\mathbf{a}^{(k)}) = 2F'_{out} \implies 2F'_{out} \text{ parameters}$$

For a single head, the number of parameters is:

$$N_{head} = F_{in}F'_{out} + 2F'_{out} = F'_{out}(F_{in} + 2)$$

So, since there are K independent heads:

$$N_{total} = K \cdot F'_{out}(F_{in} + 2) \quad (3)$$

2 Question 2

1. Let us consider the term inside the exponential of the numerator:

$$e_{ij}^{(t+1)} = \text{LeakyReLU} \left(\mathbf{a}^{(t+1)\top} \left[\mathbf{W}^{(t+1)} \mathbf{z}_i^{(t)} \parallel \mathbf{W}^{(t+1)} \mathbf{z}_j^{(t)} \right] \right)$$

Given the hypothesis $\mathbf{z}_i^{(t)} = \mathbf{z}_j^{(t)} = \mathbf{c}$, we substitute these into the equation:

$$e_{ij}^{(t+1)} = \text{LeakyReLU} \left(\mathbf{a}^{(t+1)\top} \left[\mathbf{W}^{(t+1)} \mathbf{c} \parallel \mathbf{W}^{(t+1)} \mathbf{c} \right] \right)$$

Since $\mathbf{W}^{(t+1)}$ and $\mathbf{c}$ are constant with respect to the node indices i and j , the equation becomes:

$$e_{ij}^{(t+1)} = \text{LeakyReLU} \left(\mathbf{a}^{(t+1)\top} [\mathbf{cst} \parallel \mathbf{cst}] \right)$$

All terms inside the LeakyReLU function are independent of i and j . Thus, the unnormalized attention score is a scalar constant C for all pairs of connected nodes in the graph. Indeed:

$$\begin{aligned} \alpha_{ij}^{(t+1)} &= \frac{\exp(C)}{\sum_{k \in \mathcal{N}_i} \exp(C)} \\ &= \frac{\exp(C)}{|\mathcal{N}_i| \cdot \exp(C)} \\ &= \frac{1}{|\mathcal{N}_i|} \end{aligned}$$

2. Based on the result derived in 1., the $\alpha_{ij}^{(t+1)}$ are solely determined by $(|\mathcal{N}_i|)$, ignoring the semantic content of the node features.

Consequently, the expressiveness of the GAT layer is severely reduced. The mechanism loses its ability to assign different importance weights to different neighbors based on feature similarity or relevance. It no longer behaves as an attention mechanism.

Mathematically, the update rule for node i becomes:

$$\begin{aligned} \mathbf{z}_i^{(t+1)} &= \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^{(t+1)} \mathbf{W}^{(t+1)} \mathbf{z}_j^{(t)} \right) \\ &= \sigma \left(\frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{W}^{(t+1)} \mathbf{z}_j^{(t)} \right) \end{aligned}$$

This propagation rule degenerates into a standard GCN layer.

3. The model can still perform better than random guessing by leveraging the structural information of the graph. In fact, in the Karate network, nodes belonging to the same class are more likely to be connected to each other.

Since the degenerate GAT layer acts as a standard graph convolution, it aggregates the features of a node's neighbors. Through this message-passing process, the representations of connected nodes become similar. Because connected nodes tend to share the same class, the model effectively performs smoothing over the graph topology, clustering nodes of the same community together in the embedding space solely based on their connectivity patterns, even without semantically meaningful input features.

3 Question 3

To introduce conditions into the Variational Graph Autoencoder, we need to modify the architecture so that both the encoding and decoding processes depend on an auxiliary condition vector $\mathbf{c}$.

This involves two main modifications:

1. Conditional Encoder: The encoder must learn to map the input graph and the condition to the latent space. The variational posterior becomes $q_\phi(\mathbf{z}|\mathcal{G}, \mathbf{c})$, by concatenating the condition vector $\mathbf{c}$ with the input features $\mathbf{X}$ before feeding them into the GNNs. The parameters of the Gaussian distribution are then computed as:

$$\begin{aligned} \boldsymbol{\mu} &= \text{GNN}_\mu([\mathbf{X}, \mathbf{c}], \mathbf{A}) \\ \log \boldsymbol{\sigma} &= \text{GNN}_\sigma([\mathbf{X}, \mathbf{c}], \mathbf{A}) \end{aligned}$$

2. Conditional Decoder: The generative distribution becomes $p_{\theta}(\mathcal{G}|\mathbf{z}, \mathbf{c})$. For the one-shot decoder using an MLP, we concatenate the sampled latent vector $\mathbf{z}$ with the condition vector $\mathbf{c}$ to reconstruct the adjacency matrix:

$$\hat{\mathbf{A}} = \text{MLP}([\mathbf{z} \parallel \mathbf{c}])$$

By providing $\mathbf{c}$ to both networks, the model learns to associate specific regions of the latent space with specific conditions, allowing for controlled graph generation.

4 Question 4

The reparameterization trick is necessary because the operation of sampling from a probability distribution is non-differentiable.

Training neural networks via backpropagation requires the calculation of gradients using the chain rule. If the network includes a stochastic sampling node (sampling $\mathbf{z}$ directly from $\mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$), the gradient flow is blocked at that node because there is no defined derivative for the random sampling operation with respect to the distribution parameters $\boldsymbol{\mu}$ and σ .

The trick re-expresses the random variable $\mathbf{z}$ as a deterministic function of the parameters $\boldsymbol{\mu}, \sigma$ and an independent noise variable ϵ . This restores differentiability:

$$\frac{\partial \mathbf{z}}{\partial \boldsymbol{\mu}} = 1 \quad \text{and} \quad \frac{\partial \mathbf{z}}{\partial \sigma} = \epsilon$$

allowing gradients to flow from the decoder back to the encoder.

If the model directly sampled $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$ without reparameterization, during the backward pass, the gradient of the loss function would propagate from the decoder to $\mathbf{z}$, but it would not be able to propagate further to the encoder's outputs ($\boldsymbol{\mu}$ and σ). Consequently, the weights of the encoder would receive zero gradients ($\frac{\partial \mathcal{L}}{\partial \phi} = 0$) and would never be updated. The model would effectively fail to learn the variational distribution, as the encoder and decoder would be disconnected in terms of optimization.